

Event-Time vs. Processing-Time Windowing

The Short Version

There are two different answers to "when did this happen?" — **when the event actually occurred**, and **when the system got around to processing it**. These are usually close enough together that nobody notices the difference. They stopped being close together the moment we replayed 7 days of synthetic traffic in about 20 seconds — and that gap silently broke the drift detector's windowing until we fixed it.

Two Kinds of "Time"

- **Event time**: the timestamp an event carries with it — when it actually happened, according to the data itself.
- **Processing time**: the timestamp the system stamps on it when it's actually read and handled — in our case, when Kafka records the message as ingested.

In a real, live system, these are nearly identical — a request gets processed within milliseconds of happening. It's easy to write code that (implicitly or explicitly) assumes they're interchangeable, because in normal operation, they are.

Where This Broke

Our synthetic dataset simulates 7 days of traffic, with each event carrying its own timestamp spread realistically across that week. But the **publisher script that sends this data into Kafka runs it all in one fast burst** — the entire week's worth of events gets pushed through in roughly 20 seconds, for the practical reason that nobody wants to wait a literal week to test a detector.

The consequence: every event's **processing time** (when Kafka actually received it) lands within that same 20-second window, no matter what day its **event time** claims to be from. If the drift detector's baseline job had windowed on processing time — grouping "the last 24 hours of traffic" by when Kafka saw it — it would have grouped the *entire week* into a single few-second slice, and every subsequent window would have been empty. The detector's whole notion of "a day's worth of baseline traffic" would have collapsed into noise.

The Fix

The baseline and online scoring jobs window on each event's **own event-time field** — the timestamp the synthetic generator assigned it — not on when Kafka happened to ingest it. This makes the detector's time-based logic correct regardless of how fast or slow events actually arrive, which matters for two separate reasons:

1. **Testing**: a compressed replay of historical data behaves correctly, producing the same day-by-day windows it would if actually spread across a real week.
2. **Production correctness**: if real traffic ever arrives late or out of order — a network delay, a backlog being drained — the detector still buckets it by when it actually happened, not by whenever the system happened to catch up to it.

Why This Is Worth Knowing Beyond This Project

This isn't a one-off synthetic-data quirk — it's a well-known category of bug in any streaming system (this exact distinction is core to how tools like Kafka Streams and Flink think about windowing). Any pipeline that might ever replay historical data, backfill a gap, or simply experience delayed delivery needs to make this choice deliberately. Defaulting to processing time because it's the timestamp that's "just there" is an easy, quiet way to get windowing wrong.